1

Objectives

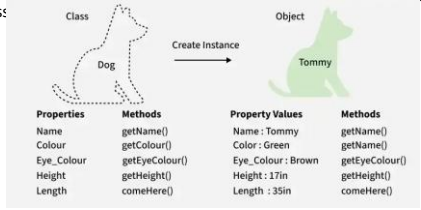
- Class and Objects
- Methods
- Constructors & Destructors
- Properties
- Encapsulation

2

Class and Objects

Introduction

- In Object-Oriented Programming, classes and objects are fundamental concepts used to represent real-world concepts and entities.
  - A class is a blueprint used to create objects with similar properties and behaviors.
  - An object is an instance of a class.
- For example, Dog is a class, while a specific dog like Tommy is an object of that class



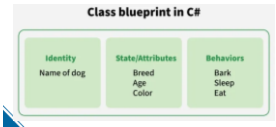
3

Class and Objects

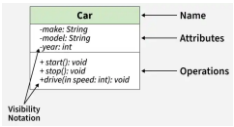
Classes

- A class is a user-defined data type that encapsulates data and behavior.
- It can contain fields, properties, methods, events, and constructors.
- A class itself does not occupy memory until its objects are created.
- Syntax in C#

```
class ClassName{
  // Fields
  // Properties
  // Methods
}
```



UML Class Notation



4

4

Class and Objects

Class declaration

- A class declaration begins with the class keyword followed by the class name. However, some optional attributes can be used with class declaration according to the application requirement. Class declarations can include these components, in order:
  - **Modifiers:** Define the accessibility of a class. By default, a class is internal.
  - **Keyword class:** Used to declare a class.
  - **Class Identifier:** The name of the class, conventionally starting with a capital letter.
  - **Base Class (Optional):** Specifies a parent class to inherit from, using the : symbol.
  - **Interfaces (Optional):** A comma-separated list of interfaces implemented by the class, also preceded by : A class can implement multiple interfaces.
  - **Body:** Enclosed within { }, containing members like fields, properties, methods, constructors and events.

```
// declaring public class
public class Sample
{
    // field variables
    public int a, b;

    // member function or method
    public void Display()
    {
        Console.WriteLine("Class in C#");
    }
}
```

5

5

Class and Objects

Objects

- An object in C# is something you create from a class, which represents a real-world entity and lets you use the data and actions defined in that class.
- An object consists of:
  - **State**: It is represented by attributes of an object and reflects the properties of an object.
  - **Behavior**: It is represented by the methods of an object and also reflects how an object interacts with other objects.
  - **Identity**: It represents the unique reference of an object, which distinguishes it from other objects.

Note: *Classes define the blueprint of state, behavior and identity. Objects are the real instances that actually hold the state, show behavior and carry identity.*

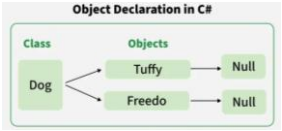
6

6

Class and Objects

Declaration of Object

- When an object is created, the class is instantiated.
  - All objects share the class's behavior, but each has its own unique state.
  - A class can have multiple instances.



- When we declare a reference variable in C# (like Dog tuffy or Dog freedo), no memory for the object is allocated at that time. The variable only holds a null reference until we explicitly create an object using the new keyword.

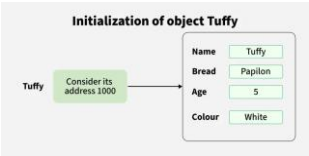
7

7

Class and Objects

Object initialization

- The ***new*** keyword instantiates a class by allocating memory for a new object and returning a reference to that memory.
- The ***new*** operator also invokes the class constructor.



- When we create an object of the Dog class and pass the parameters in the constructor. So, it allocates memory for these different objects, and each object stores its own reference in memory, and the reference variable points to that object, as shown in the image.

8

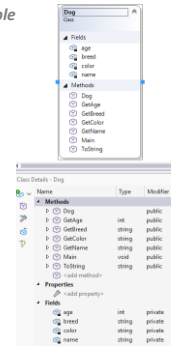
8

Class and Objects

Class, class diagram and object initialization example

```
// Class Declaration
public class Dog {
    // Instance Variables
    String name;
    String breed;
    int age;
    String color;

    // Constructor Declaration of Class same name as class
    public Dog(String name, String breed, int age, String color) {
        this.name = name;
        this.breed = breed;
        this.age = age;
        this.color = color;
    }
    public String GetName(){return name; }
    public String GetBreed()    { return breed; }
    public int GetAge()    {return age; }
    public String GetColor()    {return color; }
    public override String ToString()    {
        return "my name is: " + name +"\nmy breed is: " + breed +
"\nmy age is: " + age;
    }
}
```

```
public static void Main(String[] args)
{
    // Creating object
    Dog tuffy = new Dog("tuffy", "papillon", 5, "white");
    Console.WriteLine(tuffy.ToString());
}
```



9

9

Class and Objects

Exercises

1. Rewrite the above example.
   1. Using a UML tool (Enterprise Architect) to design the diagram.
   2. Generate C# code (or Java Code)
   3. Copy code to VS.
   4. Generate the class diagram using Class Designer tool.
   5. On Class Designer tool, add some more fields and operations.
   6. Create Main method on another class then instantiate some instances of Dog class with difference values of fields. Call methods of these instances.
2. Create a student class with arbitrary fields and methods. Create Main method to instantiate some student instances.

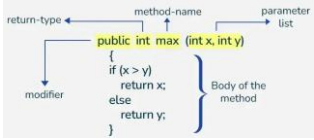
10

10

Methods

Method Declaration

- A method is a block of code that performs a specific task. It can take inputs, return results, and is defined within classes to make programs modular, readable and reusable.
- Declaring a Method: The method signature consists of the method name and the types of its parameters. It uniquely identifies the method within its class. The image below shows how a method signature is defined.



- **Access Modifier:** Defines the visibility of the method.
- **Return Type:** Specifies the type of value the method returns.
- **Method Name:** The name used to call the method.
- **Parameters:** A list of inputs the method can take and it is optional
- **Method Body:** The block of code that defines what the method does

11

11

Methods

Method Calling - Direct Method Calling

- Direct method calling occurs when you invoke a method using an instance of its class. This is the most common way to call instance methods.

```
class Sample2
{
    // Instance method to display a message
    public void DisplayMessage()
    {
        Console.WriteLine("Hello from the Display Message method!");
    }
}

class UI
{
    static void Main(string[] args)
    {
        Sample2 smp = new Sample2();
        // Create an instance of Sample2 then Call the instance method directly
        smp.DisplayMessage();
    }
}
```

12

12

Methods

Method Calling - Static Method Calling

- Static methods belong to the class itself rather than any specific instance. They can be called without creating an object of the class.

```
class Sample3
{
    // Static method to calculate square of a number
    public static int Square(int number)
    {
        // Return the square of the number
        return number * number;
    }
}

class UI2
{
    public static void Main(string[] args)
    {
        // Call static method using class name
        int result = Sample3.Square(5);

        Console.WriteLine("Square of 5 is: " + result);
    }
}
```

13

13

Methods

Method Parameters

- Methods can accept parameters to perform operations based on input values. Below is a table summarizing different types of parameters

Parameter	Description	Example
Value	Passes a copy of the argument	void Display(int x)
Reference	Passes a reference to the argument	void Update(ref int x)
Output	Parameter Used to return multiple values	void GetValues(out int x)

Instance Methods

- **Belong to an Object:** Instance methods require an object of the class to be called.
- **Access to Instance Variables:** They can access and modify instance variables and other instance methods directly.
- **Dynamic Binding:** Instance methods are shared among all objects of the class and operate on the instance-specific data.

14

14

Constructors

Introduction

- A constructor in C# is a special method of a class that is automatically called when an object of the class is created. It has the same name as the class, does not have a return type and is mainly used to initialize the object's data members.
  - A class can define multiple constructors (constructor overloading).
  - A constructor cannot be virtual or abstract. Only a special kind of constructor can be static.
- Types of Constructor
  - Default Constructor
  - Parameterized Constructor
  - Copy Constructor
  - Private Constructor
  - Static Constructor

```
class Sample5 {
    // constructor of Sample5 class
    public Sample5(){
        //...
    }
}
// Using this constructor
Sample5 obj = new Sample5();
```

15

15

Constructors

Default Constructor

- A default constructor is a constructor with no parameters.
  - It is automatically provided by the compiler if no constructor is defined in the class.
  - It initializes numeric fields to 0, Boolean to false and reference types like strings or objects to null.

```
class Sample5 {
    // Fields without manual initialization
    int num;        // default: 0
    string name;    // default: null
    bool flag;      // default: false

    // default constructor
    public Sample5(){
        Console.WriteLine("Constructor Called");
    }
}
class UI5
{
    public static void Main(string[] args)
    {
        // Create an instance of Sample5
        Sample5 sample = new Sample5();
    }
}
```

16

16

Constructors

Parameterized Constructor

- A constructor having at least one parameter is called a parameterized constructor.
- It can use to initialize each instance of the class to different values.

```
class Sample6 {
    // store name value
    String n;
    // store Id
    int i;
    // the values of passed arguments while object of that class created.
    public Sample6(String n, int i) {
        this.n = n;
        this.i = i;
    }
    public String ToString() {
        return "Name = " + n + " Id = " + i;
    }
}
class UI6 {
    public static void Main(string[] args) {
        // Create an instance of Sample6, It will invoke parameterized constructor
        Sample6 sample = new Sample6("John", 123);
        Console.WriteLine(sample);
    }
}
```

17

17

Constructors

Copy Constructors

- A copy constructor is used to create a new object by copying values from an existing object of the same class.
- Copy constructors are not built-in and must be explicitly defined by the user.

```
class Sample7 {
    private string month;
    private int year;

    // declaring Copy constructor
    public Sample7(Sample7 s) {
        month = s.month;
        year = s.year;
    }

    // Instance constructor
    public Sample7(string month, int year) {
        this.month = month;
        this.year = year;
    }

    // Get details of Sample7 object
    public string Details {
        get {
            return "Month: " + month.ToString() + "\nYear: " + year.ToString();
        }
    }
}
```

```
public static void Main() {
    // Create a new Sample7 object.
    Sample7 g1 = new Sample7("June", 2026);

    // here is g1 details is copied to g2.
    Sample7 g2 = new Sample7(g1);

    Console.WriteLine(g2.Details);
}
```

18

18

Constructors

Private Constructors

- If a constructor is created with a **private** specifier is known as Private Constructor. It is not possible for other classes to derive from this class and also, it's not possible to create an instance of this class. Some important points regarding the topic is mentioned below:
  - A private constructor is often used in implementing the Singleton design pattern, but it does not implement the pattern by itself.
  - Use a private constructor when we have only static members.
  - Using a private constructor prevents the creation of the instances of that class.
- ***Note:*** *Access modifiers can be used in constructor declaration to control its access i.e. which other class can call the constructor. Private Constructor is one of it's example.*

```
class Sample8 {
    // private Constructor
    private Sample8() {
        Console.WriteLine("from private constructor");
    }

    public static int count;

    public static int Count() {
        return ++count;
    }
}
```

```
public static void Main() {
    // This will cause an error (private constructor),
// Sample8 s = new Sample8();
    // Accessing without any instance of the class
    Sample8.Count();
}
```

19

19

Constructors

Static Constructors

- Static Constructor has to be invoked *only once* in the class, and it has been invoked during the creation of the first reference to a static member in the class. A static constructor is used to initialize static fields or data of a class and is executed only once.
  - It can't be called directly.
  - When it is executing then the user has no control.
  - It does not take access modifiers or any parameters.
  - It is called automatically to initialize the class before the first instance is created.

```
class Sample9 {
    // It is invoked before the first instance constructor is run.
    static Sample9() {
        Console.WriteLine("Static Constructor");
    }

    public Sample9(int i) {
        Console.WriteLine("Instance Constructor " + i);
    }

    public string display_details(string name, int id) {
        return "Name: " + name + " id: " + id;
    }
}
```

```
public static void Main() {
    // Here both Static and instance constructors are invoked for
first instance
    Sample9 obj = new Sample9(1);

    Console.WriteLine(obj.display_details("GFG", 1));

    Sample9 obj1 = new Sample9(2);

    Console.WriteLine(obj1.display_details("GeeksForGeeks", 2));
}
```

20

20

Destructors

Introduction

Syntax

```
class ClassName {
// Destructor
~ClassName() {
// Cleanup code
}
}
```

- A destructor in C# is a special method that is automatically called when an object is destroyed or removed from memory.
- It is used to release unmanaged resources like file handles, database connections or network streams before the object is reclaimed by the garbage collector.
  - Declared using a tilde (~) followed by the class name.
  - Cannot have access modifiers or parameters.
  - A class can have only one destructor.
  - Cannot be called explicitly, it is invoked automatically by the garbage collector.
  - Useful for cleaning up unmanaged resources, but not necessary for managed memory.
  - It cannot be defined in Structures. It is only used with classes.
  - It cannot be overloaded or inherited.
  - It is called when the garbage collector executes finalization, not necessarily when the program exits.
  - Internally, a destructor is translated by the compiler into an override of the Finalize() method.

21

21

Destructors

Example

```
class Sample10
{
    // Constructor
    public Sample10()
    {
        Console.WriteLine("Object Created.");
    }

    // Destructor
    ~Sample10()
    {
        Console.WriteLine("Object Destroyed.");
    }

    public void DisplayMessage()
    {
        Console.WriteLine("Message Printed.");
    }

    public static void Main(string[] args)
    {
        // Create an instance of Sample10
        Sample10 g = new Sample10();

        // Destructor will be called when g goes out of scope
        g.DisplayMessage();
    }
}
```

```
Object Created.
Message Printed.
Object Destroyed.
```

22

22

Methods, Constructors & Destructors

Exercises

Problem Statement: Write a C# class named Student that satisfies the following requirements:

- **1. Fields**
  - name (string) — the student's name
  - score (double) — the student's average score
  - static int totalStudents — a static field that counts the total number of Student objects created
- **2. Constructor**
  - Initialize name and score, and **increment totalStudents by 1** every time a new Student object is created.
- **3. Instance Methods (require this, operate on a specific object)**
  - GetName() — returns the student's name
  - GetScore() — returns the student's score
  - IsPassed() — returns true if score >= 5.0, otherwise false
  - GetClassification() — classifies the student based on their score:
    - score >= 8.0 → "Excellent"
    - score >= 6.5 → "Good"
    - score >= 5.0 → "Average"
    - otherwise → "Weak"
- **4. Static Methods (do NOT use this, operate on shared data or parameters passed in)**
  - static int GetTotalStudents() — returns the total number of students created
  - static Student FindTopStudent(Student[] students) — takes an array of students and returns the one with the highest score
  - static double CalculateAverageScore(Student[] students) — calculates the average score of the entire list
- **5. Main Program**

Read full on word document...

23

23

á

Properties

Introduction

- A property in C# is a member of a class that provides a flexible way to read, write or compute the value of a private field. It acts like a combination of a variable and a method.
  - Declared inside a class using { get; set; }.
  - **get**: returns the value of the field.
  - **set**: assigns a value to the field.
  - Can be read only (get only) or write only (set only).
  - Improves encapsulation by controlling access to fields.

```
class People
{
    private int age;
    public int Age
    {
        get { return age; }
        set
        {
            if (value >= 0) age = value;
            else Console.WriteLine("Age cannot be
negative!");
        }
    }
}
```

- There are two main reasons to use properties:
  - To provide controlled and secure access to private data members, including validation and logic through accessors.
  - To protect members of the class so another class may not misuse those members.

24

24

Properties

Accessors

- The block of "set" and "get" is known as Accessors.
- They can be used to restrict or control accessibility based on design requirements.
- There are two types of accessors: get accessors and set accessors.

```
class C1
{
    // Public data members (No restrictions)
    public int rn;
    public string name;

    // Private field (Cannot be accessed directly)
    private int marks = 35;

}
```

Problem Without Properties

```
public class Sample11
{
    public static void Main(string[] args) {
        // Creating an object of class C1
        C1 obj = new C1();
        // Setting values to public data members
        obj.rn = 10080;
        obj.name = null;
        // Directly modifying private field is not possible
        obj.marks = 0; // Error
        Console.WriteLine("Name: {0} \nRoll No: {1}", obj.name, obj.rn);
    }
}
```

25

25

Properties

Declaration Syntax

- Syntax

```
<access_modifier> <return_type> <property_name>{
        get { // body }
        set { // body }
}
```

- These are the different terms used to define the properties
  - **Access-modifiers:** Used to define the accessibility levels. It can be public, private, protected or internal.
  - **Return-type:** Used to define which value returns with the end of the method.
  - **Get accessor:** Used to get the values.
  - **Set accessor:** Used to set or modify the values.
- Types of Properties
  - **Read and Write Properties**: When the property contains both get and set methods.
  - **Read-Only Properties**: When the property contains only the get method.
  - **Write Only Properties**: When the property contains only a set method.
  - **Auto-Implemented Properties**: When there is no additional logic in the property accessors it is introduced in C# 3.0.

26

26

Properties

Get Accessor (Read-only Property)

- It specifies that the value of a field can be accessed publicly. It returns a single value, and it specifies the read-only property.

```
class Sample12
{
    // Private field
    private int r = 357;

    // Read-only property
    public int Roll_no
    {
        get { return r; }
    }
    public static void Main(String[] args)
    {
        Sample12 o = new Sample12();
        Console.WriteLine("Roll no: " + o.Roll_no);
    }
}
```

27

27

Properties

Set Accessor (Write-Only Property)

- It will specify the assignment of a value to a private field in a property. It returns a single value, and it specifies the write-only property.

```
1 reference
class Sample13
{
    // Private field
    private int r = 357;
    1 reference
    public Sample13(int rollNo) { this.Roll_No = rollNo; }

    /// Read-Write Property
    6 references
    public int Roll_No
    {
        get { return r; }
        set { r = value; }
    }
    0 references
    public static void Main(String[] args)
    {
        Sample13 o = new Sample13(107);
        Console.WriteLine($"Current Roll No: {o.Roll_No}");

        // Modify RollNo using setter
        o.Roll_No = 357;
        Console.WriteLine($"Changed Roll No: {o.Roll_No}");
    }
}
```

What happen if we disable loc #16?

```
public int Roll_No
{
    //get { return r; }
    set { r = value; }
}
```

28

28

Properties

Auto-implemented property

```
public class Student
{
    // Auto-implemented property
    public string Name { get; set; } = "Sample";
}
class Sample14
{
    public static void Main(string[] args)
    {
        Student s = new Student();

        Console.WriteLine("Name: " + s.Name);

        // Changing property value
        s.Name = "Change Name using Auto-implemented property ";
        Console.WriteLine("Changed Name: " + s.Name);
    }
}
```

29

29

- **Exercise: The UserAccount Management System:** You are building the core backend module for a financial app. You need to construct a UserAccount class that encapsulates user data safely by using C# properties with various access levels and business logic rules.
- **Requirements:** Implement the UserAccount class with the following properties:
- **AccountId** *(Init-Only Property)*
  - **Type:** string
  - **Behavior:** Can be read freely, but can **only** be set during object creation (using C# init).
- **Username** *(Auto-Implemented Property)*
  - **Type:** string
  - **Behavior:** Standard read and write access (get; set;).
- **Password** *(Write-Only Property)*
  - **Type:** string
  - **Behavior:** Can **only** be set, never read (set only). When set, it should prepend "[ENCRYPTED]_" to the input string and store it in a private field _password.
- **Balance** *(Full Property with Validation)*
  - **Type:** decimal
  - **Behavior:** Uses a private backing field _balance.
    - **Get:** Returns _balance.
    - **Set:** Ensures the new value is **not negative**. If a negative value is assigned, print "Error: Balance cannot be negative!" to the console and keep the previous balance intact.
- **IsVIP** *(Computed / Expression-Bodied Read-Only Property)*
  - **Type:** bool
  - **Behavior:** Has only a getter (get => ...). Dynamically returns true if Balance is **$10,000 or greater**, otherwise false.
- **CreatedDate** *(Get-Only Auto Property)*
  - **Type:** DateTime
  - **Behavior:** Can only be read. Initialize it to DateTime.Now inside the class constructor so it cannot be altered afterward.

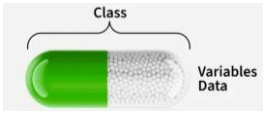
30

30

Encapsulation

Introduction

- Encapsulation is one of the core principles of object-oriented programming (OOP).
- It refers to the practice of binding data (fields) and the methods that operate on that data into a single unit, while restricting direct access to some components.
- This ensures controlled interaction with an object's internal state.



31

31

Encapsulation

Key Concepts

- Encapsulation hides the internal representation of an object and exposes only necessary operations.
- Fields are often kept private while access is provided through public properties or methods.
- It improves data security, code maintainability and flexibility.
- Access modifiers (private, public, protected, internal) control visibility of members.

32

32

Encapsulation

Visibility Modifiers

Modifier	Same class	Subclass	Same Assembly (DLL)	Outside Assembly
private	Yes	No	No	No
protected	Yes	Yes	No	No
internal	Yes	Yes	Yes	No
protected internal	Yes	Yes	Yes	Yes (Outside subclass)
public	Yes	Yes	Yes	Yes

33

33

Encapsulation

Encapsulation Using constructor, method

```
class Account {
    private double balance; // hidden field

    public void Deposit(double amount) {
        if (amount > 0)
            balance += amount;
    }

    public void Withdraw(double amount) {
        if (amount <= balance)
            balance -= amount;
    }

    public double GetBalance() {
        return balance;
    }
}
```

```
class Sample15
{
    static void Main()
    {
        Account acc = new Account();
        acc.Deposit(500);
        acc.Withdraw(200);
        Console.WriteLine("Balance: " + acc.GetBalance());
    }
}
```

34

34

Encapsulation

Encapsulation Using Properties

- Properties can be used to simplify encapsulation, acting like smart getters and setters.

```
class Student1 {
    private string name; // private field

    public string Name {
        get { return name; }
        set {
            if (!string.IsNullOrEmpty(value))
                name = value;
        }
    }
}
```

Using Exception handling instead?!

```
class Sample16 {
    static void Main() {
        Student1 s = new Student1();
        s.Name = "John Doe";
        Console.WriteLine("Name: " + s.Name);
    }
}
```

35

35

Encapsulation

Exercise

- **Scenario:** You are developing a secure banking module. Direct access to sensitive internal data (like balance, account status, or PIN) must be blocked. External code should only interact with the account using well-defined methods that enforce rules (e.g., PIN checks, balance limits, account lockout after failed attempts).
- **Requirements**
  - Implement the BankAccount class using proper encapsulation principles:
- **Private Encapsulated State**
  - _balance (decimal): Private backing field. Must never be modified directly from outside the class.
  - _pin (string): Private backing field containing a 4-digit PIN code.
  - _failedAttempts (int): Private counter tracking consecutive wrong PIN entries.
- **Controlled Properties**
  - **AccountHolder** (string): Public read-only property (set via constructor only).
  - **IsLocked** (bool): Public getter, private set. Returns true if _failedAttempts >= 3.
- **Encapsulated Methods & Business Logic**
  - **Deposit(decimal amount)**:
    - Accepts only positive amounts (> 0). If invalid, print an error message and return false.
    - If valid, add to _balance and print success message.
  - **Withdraw(decimal amount, string inputPin)**:
    - Fails if IsLocked is true.
    - Validates inputPin. If incorrect, increment _failedAttempts, print an error, lock account if attempts reach 3, and return false.
    - If PIN is correct, reset _failedAttempts to 0.
    - Checks if amount is positive and _balance >= amount. If insufficient, print error. Otherwise, deduct from _balance.
  - **GetBalance(string inputPin)**:
    - Balance view is protected! Returns _balance **only if** inputPin is correct. If PIN fails, print error and return -1m.
  - **ChangePin(string currentPin, string newPin)**:
    - Requires valid currentPin.
    - Validates that newPin is non-null, exactly 4 characters long, and numeric. If valid, update _pin.

36

36

SOLID Principles

Single Responsibility Principle (SRP)

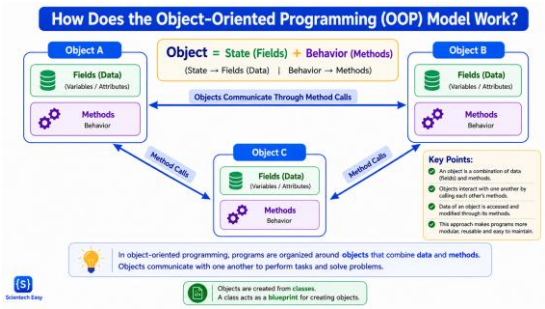
- A class should have only one reason to change, meaning that it should have only one responsibility or purpose.

```
// ❌ Violate SRP: Order class has multiple responsibilities (saving to database and sending email)
public class Order {
    public void SaveToDatabase() { /* SQL */ }
    public void SendEmailInvoice() { /* SMTP */ }
}

// ✅ Separate responsibilities into different classes
public class Order { /* Chỉ chứa dữ liệu đơn hàng */ }

public class OrderRepository { public void Save(Order o) { } }

public class EmailService { public void Send(Order o) { } }
```

37

37

Key issue



Source: https://www.scientecheasy.com/

38

38

References

1. https://www.geeksforgeeks.org, accessed 08/2026
2. https://www.scientecheasy.com/, accessed 08/2026

39

39

40